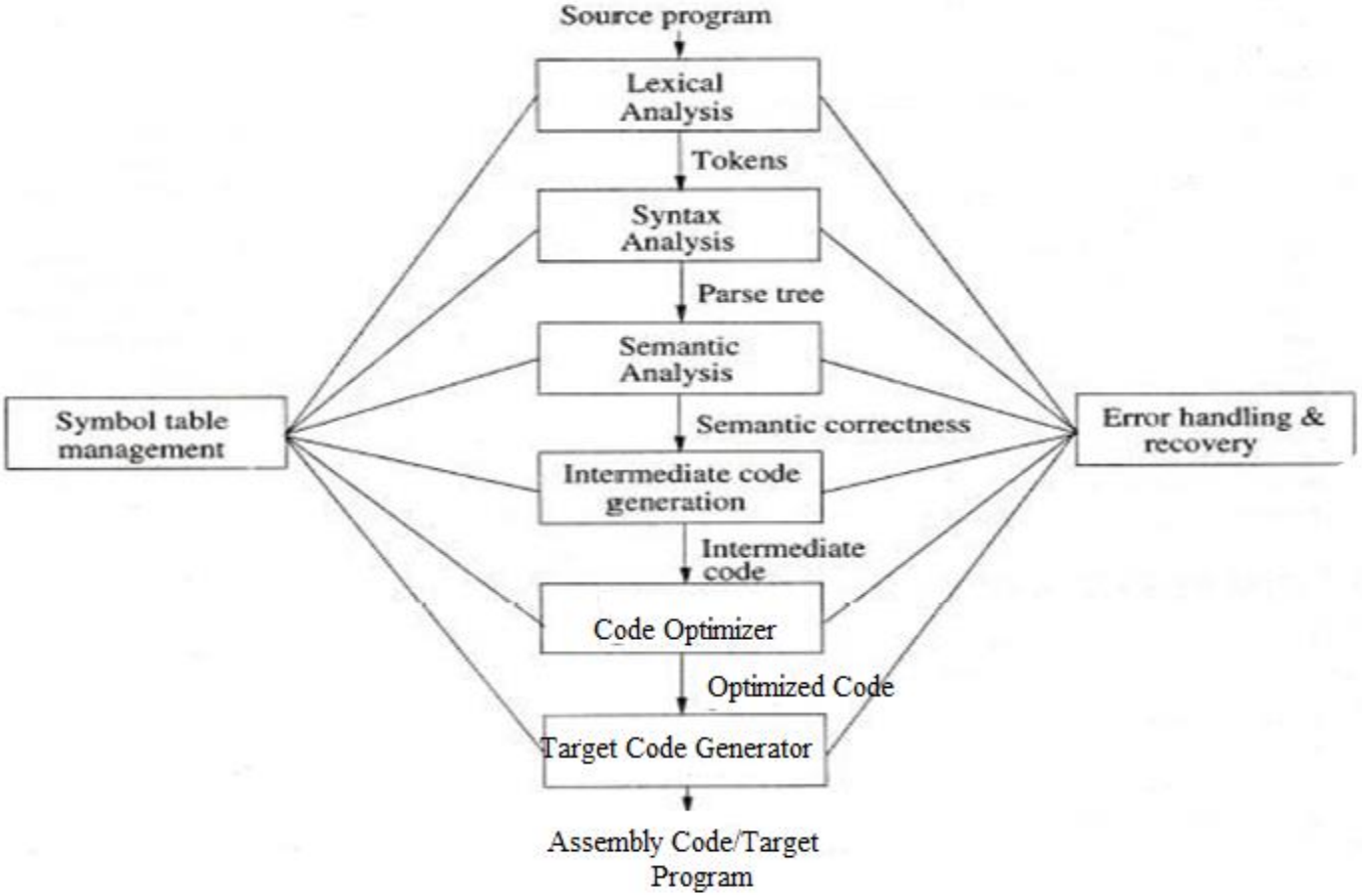
Phases of a Compiler



Lexical Analysis

- The lexical analyzer reads the characters in the source program and groups them into a stream of **tokens**.
- Also removes cosmetics, like extra white spaces, comments etc. from the program
- When an identifier such as **rate** is found, the lexical analyzer generates an **id** token and also enters the lexeme **rate** into the **symbol table**.

Example

Source statement:

position := initial + rate * 60

Token stream:

id1 := id2 + id3 * 60

where:

id1 → position

id2 → initial

id3 → rate

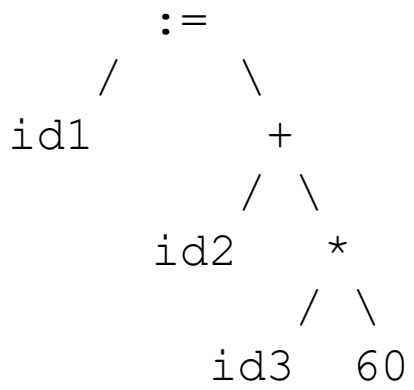
Syntax Analysis

- The **Syntax Analyzer (Parser)** receives the stream of tokens from the lexical analyzer and checks whether the token sequence follows the grammar (CFG) of the programming language.
- If the input is syntactically correct, it constructs a **Syntax Tree (Parse Tree)**.
- Error messages are flashed for syntactically incorrect program

For the Token stream:

id1 := id2 + id3 * 60

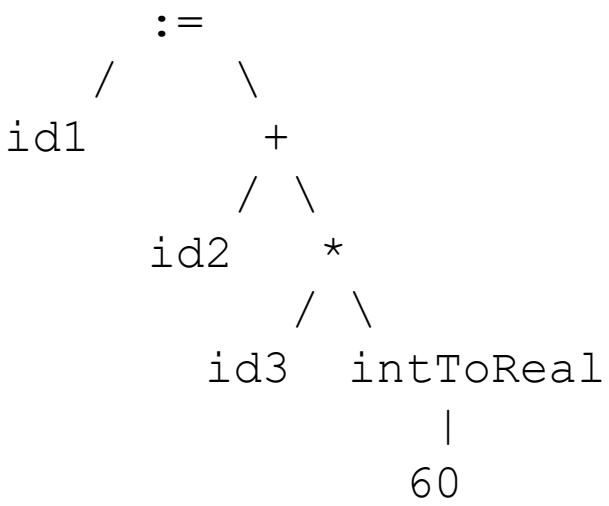
Syntax Tree:



Semantic Analysis

- The **Semantic Analyzer** checks the meaning of the program after syntax analysis.
- It performs tasks such as:
  - Type checking
  - Scope checking
  - Declaration checking
  - Compatibility of operands and operators
- During semantic analysis, constants are assigned their data types.

Example:



Here, the constant **60** is converted (if required) to the appropriate type (e.g., **intToReal**) to make the expression type-compatible.

Intermediate Code Generation

Compilers generate an **Intermediate Representation (IR)** of the source program. This intermediate representation should have two important properties:

1.It should be easy to produce.

2.It should be easy to translate into the target program (machine code).

Example

Source statement:

position := initial + rate * 60

Intermediate Representation (IR) of the source program:

```
temp1 := intToReal(60)
temp2 := id3 * temp1
temp3 := id2 + temp2
id1   := temp3
where:
  id1 → position
  id2 → initial
  id3 → rate
```

Three-Address Code (TAC)

The intermediate representation can have a variety of forms. One of the most commonly used forms is **Three-Address Code (TAC)**.

Definition:

Three-address code consists of a sequence of instructions, where **each instruction contains at most three operands**. Typical instruction formats are:

$$x := y \text{ op } z$$

$$x := \text{op } y$$

$$x := y$$

Code Optimization

The **Code Optimization** phase attempts to improve the intermediate code so that the generated machine code executes **faster** and/or uses **fewer resources**, without changing the program's meaning.

Optimized Intermediate Code:

Instead of:

```
temp1 := intToReal(60)
temp2 := id3 * temp1
temp3 := id2 + temp2
id1   := temp3
```

The optimized code can become:

```
temp1 := id3 * 60.0
id1   := id2 + temp1
```

This version uses fewer temporary variables and fewer instructions while producing the same result.

Code Generation

Code Generation is the **final phase of the compiler**. In this phase, the compiler generates the **target code.**
During code generation:

- Memory locations are selected for each variable used in the program.
- Each intermediate-code instruction is translated into one or more machine instructions.
- The generated machine instructions perform the same task as the intermediate code.

Example

Consider the optimized intermediate code:

```
temp1 := id3 * 60.0
id1   := id2 + temp1
```

Using registers **R1** and **R2**, the corresponding target code may be:

```
MOVF  id3, R2
MULF  #60.0, R2
MOVF  id2, R1
ADDF  R2, R1
MOVF  R1, id1
```

The **first operand** of each instruction specifies the **source**. The **second operand** specifies the **destination**.

The suffix **F** in MOVF, MULF, and ADDF indicates that the instructions operate on **floating-point values**.
The symbol # denotes an **immediate constant**. Thus, #60.0 means that 60.0 is treated as a constant .

Symbol Table Management

An essential function of a compiler is to **record the identifiers** used in the source program and **collect information about the various attributes** of each identifier.

These attributes provide information such as:

- Storage location allocated for the identifier
- Data type of the identifier
- Scope of the identifier (where in the program it is valid)
- Other properties required during compilation

Symbol Table

A **Symbol Table** is a data structure that contains a record for each identifier in the source program.

Each record stores the attributes associated with an identifier. The data structure allows the compiler to:

- Quickly locate the record for an identifier.
- Efficiently store new information.
- Retrieve identifier information whenever required by different phases of the compiler.

Example of Symbol Table

For the statement:
position := initial + rate * 60.0

Index	Lexeme	Token	Data Type	Scope	Memory Address
1	position	id	float	Global	1000
2	initial	id	float	Global	1004
3	rate	id	float	Global	1008

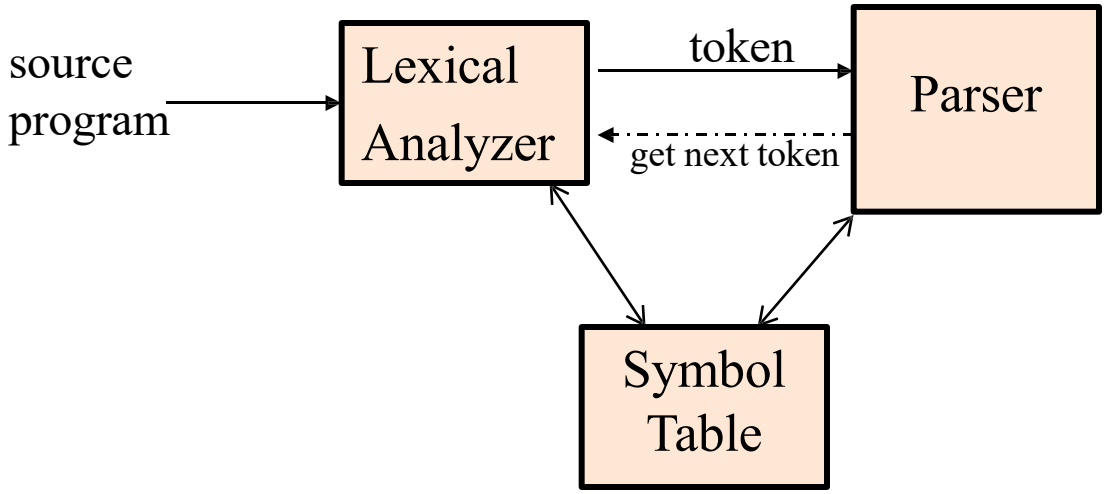
Error Detection and Reporting

Each phase of a compiler can encounter errors. The **syntax analysis** and **semantic analysis** phases usually detect a large fraction of the errors that can be identified by the compiler.

- The **lexical analysis** phase detects errors when the sequence of characters in the input does not form valid tokens of the language.
- Errors in which the token stream violates the grammatical (syntax) rules of the language are detected during **syntax analysis**.
- During **semantic analysis**, the compiler detects constructs that have the correct syntactic structure but do not have a valid meaning.

LEXICAL ANALYZER

- **Lexical Analyzer** reads the source program character by character to produce **tokens**.
- Normally a lexical analyzer doesn't return a list of tokens at one shot, it returns a token when the parser asks a token from it.



TOKENS, LEXEMES , AND PATTERNS

- A **lexeme** is the **actual sequence of characters** in the source program that matches a token pattern.
- A **token** is a **category or class** assigned to a lexeme. In other words, A token is a symbolic name representing a class of lexemes with similar characteristics.
- A **pattern** describes the rule used to recognize a token.

Lexeme	**Pattern (in words)**	**Token**
int	Reserved keyword	Keyword
count	Begins with a letter and may contain letters and digits	id